



Shared Preferences

Corso di Laboratorio di Programmazione per Sistemi Mobile e Tablet
2025/2026

Il problema della persistenza

Quando l'utente chiude un'app, la RAM viene liberata. Le variabili in memoria spariscono. Se vogliamo ricordare qualcosa fra un'esecuzione e la successiva, dobbiamo scriverla su disco.



SharedPreferences

Coppie chiave-valore in un file XML.
Pochi dati, lettura/scrittura sincrona o asincrona.

Esempio:

preferenze utente, flag, token semplici



File su disco

File di testo o binari nella memoria interna o esterna dell'applicazione.

Esempio:

documenti, immagini, cache di grandi dimensioni



Database (Room/SQLite)

Tabelle relazionali, query SQL, supporto a relazioni e indici.

Esempio:

elenchi grandi e strutturati, dati relazionali

In questa lezione ci concentriamo sulla prima opzione.

Cosa sono le SharedPreferences

Una SharedPreferences è un'interfaccia fornita dal sistema Android per memorizzare in modo persistente piccole quantità di dati primitivi sotto forma di coppie chiave-valore.

Punti chiave da ricordare

- Persistente: i dati sopravvivono alla chiusura dell'app e al riavvio del dispositivo.
- Privata di default: il file appartiene all'app e non è leggibile dalle altre applicazioni.
- Coppie chiave-valore: ogni dato è identificato da una stringa univoca (la chiave).
- Tipi supportati limitati: String, Int, Long, Float, Boolean, Set<String>.
- Pensata per piccole quantità di dati (decine o centinaia di valori, non migliaia).
- Memorizza in un file XML nella cartella privata dell'applicazione.



Non sono il posto giusto per password, immagini, liste lunghe o dati sensibili in chiaro.

Tipi di dato supportati

Le SharedPreferences supportano un insieme limitato di tipi primitivi più una collezione: il `Set<String>`. Per ogni tipo esiste un metodo `putXxx()` per scrivere e `getXxx()` per leggere.

Tipo Kotlin	putXxx	getXxx	Esempio
String	putString	getString(key, default)	nome utente, email, token
Int	putInt	getInt(key, default)	contatore, livello, età
Long	putLong	getLong(key, default)	timestamp, identificatori
Float	putFloat	getFloat(key, default)	punteggio, soglia, fattore
Boolean	putBoolean	getBoolean(key, default)	tema scuro on/off, primo avvio
Set<String>	putStringSet	getStringSet(key, default)	tag selezionati, città preferite



Manca il tipo che ti serve?

Per Date usa il timestamp Long; per oggetti complessi serializza in JSON e salva la String.

Ottenere l'istanza di SharedPreferences

Esistono due modi principali per ottenere un'istanza di SharedPreferences. Il primo usa un nome di file scelto da noi, il secondo usa il file di default.

`getSharedPreferences(name, mode)`

File con nome scelto da noi. Utile quando vogliamo separare gruppi di preferenze (es. "user_prefs", "app_state").

```
val prefs = context.getSharedPreferences(  
    "user_prefs",  
    Context.MODE_PRIVATE  
)
```

Kotlin

`PreferenceManager.getDefaultSharedPreferences(context)`

File di default usato dalle PreferenceFragment / impostazioni di sistema. Nome generato dal framework.

```
import androidx.preference.PreferenceManager  
  
val prefs = PreferenceManager  
    .getDefaultSharedPreferences(context)
```

Kotlin

Il parametro mode

Il secondo parametro di `getSharedPreferences` indica come il file viene esposto ad altre app. Oggi l'unico valore valido è `MODE_PRIVATE`.

USA QUESTO

MODE_PRIVATE

Il file è leggibile e scrivibile solo dall'app stessa. È l'unico mode da usare oggi.

DEPRECATO · NON USARE

MODE_WORLD_READABLE / WORLD_WRITEABLE

Permettevano la lettura/scrittura da altre app. Deprecati da Android 4.2 (API 17), causa di vulnerabilità note.

```
val prefs = getSharedPreferences("user_prefs", Context.MODE_PRIVATE) // unica scelta sensata
```

Kotlin

Leggere un valore con getXxx()

Tutti i metodi di lettura accettano un valore di default. Se la chiave non esiste o non è del tipo richiesto, viene restituito il default.

```
val prefs = context.getSharedPreferences(  
    "user_prefs", Context.MODE_PRIVATE  
)  
  
// Stringa: il default può essere null  
val nome: String? =  
    prefs.getString("username", null)  
  
// Boolean: default tipico false  
val darkMode: Boolean =  
    prefs.getBoolean("dark_mode", false)  
  
// Int: default 0  
val livello: Int = prefs.getInt("livello", 0)  
  
// Long: utile per timestamp  
val ultimoLogin: Long =  
    prefs.getLong("last_login", 0L)
```

Kotlin

1

Il default è il secondo argomento di ogni getXxx.

2

Per le String il default può essere null (firma getString(key, defValue: String?)).

3

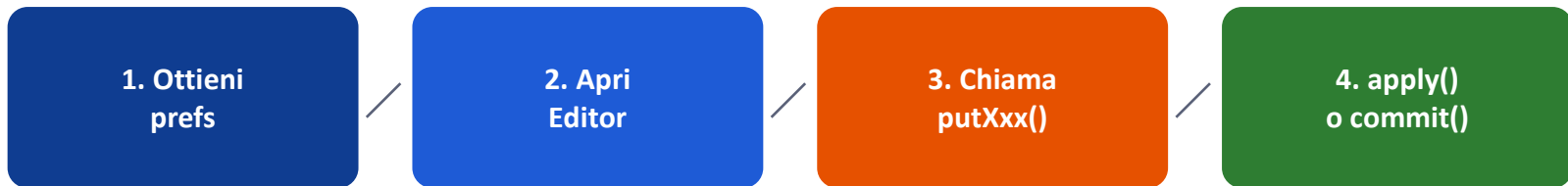
Le letture sono sincrone: il valore è già pronto perché caricato in memoria.

4

Una stessa istanza di prefs è un singleton condiviso: thread-safe per le letture.

Scrivere: l'oggetto Editor

Per modificare le preferenze non si chiama `putXxx()` direttamente sull'istanza, ma su un Editor temporaneo, ottenuto con `edit()`. Tutte le modifiche sono in transazione.



```
val prefs = context.getSharedPreferences("user_prefs", Context.MODE_PRIVATE)
val editor = prefs.edit() // 2. ottiene un Editor
editor.putString("username", "alice") // 3. accumula modifiche
editor.putInt("livello", 5)
editor.putBoolean("dark_mode", true)
editor.apply() // 4. scrive tutto in un colpo solo
```

Kotlin

Stile idiomatico Kotlin: edit { }

Il modulo `androidx.core.content` fornisce un'estensione che rende il codice più conciso: non serve più ottenere l'editor manualmente.

Stile Java-like (verboso)

```
val editor = prefs.edit()
editor.putString(
    "username", "alice"
)
editor.putInt("livello", 5)
editor.putBoolean(
    "dark_mode", true
)
editor.apply()
```

Kotlin

Stile Kotlin idiomatico

```
import androidx.core.content
    .edit

prefs.edit {
    putString(
        "username", "alice"
    )
    putInt("livello", 5)
    putBoolean(
        "dark_mode", true
    )
}
```

Kotlin



`edit { }` chiama `apply()` in automatico. Per usare `commit()` invece, passare `commit = true`: `prefs.edit(commit = true) { ... }`.

apply() vs commit(): la differenza chiave

Esistono due modi per chiudere una transazione di scrittura. La scelta sbagliata può degradare la fluidità dell'app.

Aspetto	apply()	commit()
Tipo di scrittura	Asincrona (in background)	Sincrona (blocca il chiamante)
Valore di ritorno	void (nulla)	Boolean (esito)
Memoria in-RAM	Aggiornata immediatamente	Aggiornata immediatamente
Disco	Scritto in differita	Scritto prima del ritorno
Quando usarlo	99% dei casi	Solo se serve l'esito immediato
Thread	Non blocca il main thread	Blocca il chiamante: NO su main



Regola pratica: usare SEMPRE apply(). Usa commit() solo se l'esito o la sincronia sono indispensabili.

Esempio CRUD completo

Mettiamo insieme lettura e scrittura in una piccola Activity che ricorda l'username e il tema preferito.

```
class MainActivity : AppCompatActivity() {  
    private val prefs by lazy {  
        getSharedPreferences("user_prefs", Context.MODE_PRIVATE)  
    }  
    override fun onCreate(state: Bundle?) {  
        super.onCreate(state)  
        setContentView(R.layout.activity_main)  
  
        // LETTURA: ripristino lo stato salvato  
        val savedName = prefs.getString("username", "")  
        val darkMode = prefs.getBoolean("dark_mode", false)  
        binding.nomeInput.setText(savedName)  
        binding.temaSwitch.isChecked = darkMode  
  
        // SCRITTURA: salvo quando cambia qualcosa  
        binding.salvaBtn.setOnClickListener {  
            prefs.edit {  
                putString("username", binding.nomeInput.text.toString())  
                putBoolean("dark_mode", binding.temaSwitch.isChecked)  
            }  
        }  
    }  
}
```

Kotlin

remove, clear, contains

Oltre a leggere e scrivere, l'API offre operazioni di rimozione e ispezione delle chiavi.

```
// Verifica se una chiave esiste
val haGiaSalvato = prefs.contains("username")

// Rimuove una sola chiave
prefs.edit {
    remove("username")
}

// Cancella TUTTE le chiavi del file
prefs.edit {
    clear()
}

// Combina clear() e altre put()
// clear() viene applicato per primo
prefs.edit {
    clear()
    putBoolean("first_run", true)
}
```

Kotlin

contains(key)

ritorna Boolean. Utile per distinguere "chiave assente" da "valore = default".

remove(key)

rimuove una sola coppia. Ricorda comunque apply() o commit().

clear()

rimuove TUTTE le chiavi del file. Operazione distruttiva.

Ordine garantito

in una stessa edit() clear() viene eseguito prima di tutti i put().

getAll(): leggere tutte le chiavi

Per scopi di debug, di esportazione o di backup è possibile ottenere tutte le coppie chiave-valore in una mappa.

```
// Ritorna Map<String, *> non modificabile
val tutte: Map<String, *> = prefs.all

// Iterazione semplice
for ((chiave, valore) in tutte) {
    Log.d("Prefs", "$chiave = $valore")
}

// Trasformazione in JSON per debug
val json = tutte.entries
    .joinToString(", ") {
        "\"${it.key}\"": ${it.value}"
    }
Log.d("Prefs", "{ $json }")

// Conteggio
Log.d("Prefs", "Totale: ${tutte.size}")
```

Kotlin

Quando usarlo?

- Debug e log durante lo sviluppo
- Esportare lo stato in un file di backup
- Migrare dati da una versione all'altra
- Test automatici di stato



Map<String, *> = i tipi non sono garantiti, usare cast con prudenza.

Reagire ai cambiamenti: il Listener

Possiamo essere notificati ogni volta che una chiave viene modificata, registrando un `OnSharedPreferencesChangeListener`.

```
class MainActivity : AppCompatActivity() {  
    private val listener =  
        SharedPreferences.OnSharedPreferencesChangeListener { sp, key ->  
            when (key) {  
                "dark_mode" -> applicaTema(sp.getBoolean(key, false))  
                "username" -> aggiornaIntestazione(sp.getString(key, ""))  
            }  
        }  
    override fun onResume() {  
        super.onResume()  
        prefs.registerOnSharedPreferencesChangeListener(listener)  
    }  
    override fun onPause() {  
        super.onPause()  
        prefs.unregisterOnSharedPreferencesChangeListener(listener)  
    }  
}
```

Kotlin



ATTENZIONE: salvare il listener in una proprietà (val), altrimenti il GC può rimuoverlo (è registrato con WeakReference).

Dove finiscono i dati sul dispositivo

Le SharedPreferences vivono in un file XML nella cartella privata dell'app, accessibile solo dal sistema e dall'app stessa.

Path sul device

```
/data/data/  
└─ com.example.miaapp/  
    │   └─ shared_prefs/  
    │       │   └─ user_prefs.xml  
    │       │   └─ app_state.xml  
    │   └─ databases/  
    │   └─ files/  
    └─ cache/
```

Cose da ricordare

- Cartella privata: nessun'altra app ci accede.
- Un file XML per ogni nome scelto.
- Cancellando i dati dell'app il file viene rimosso.
- Il backup automatico Android li include di default.
- Su emulatore/root puoi ispezionarli con adb.

Com'è fatto il file XML

Dato il codice di scrittura mostrato a sinistra, sul disco viene salvato il file XML mostrato a destra.

Codice Kotlin

```
prefs.edit {  
    putString("username", "alice")  
    putInt("livello", 5)  
    putBoolean("dark_mode", true)  
    putFloat("volume", 0.75f)  
    putStringSet(  
        "tag",  
        setOf("kotlin", "android")  
    )  
}
```

Kotlin

File user_prefs.xml risultante

```
<?xml version='1.0' encoding='utf-8'?>  
<map>  
    <string name="username">alice</string>  
    <int name="livello" value="5"/>  
    <boolean name="dark_mode" value="true"/>  
    <float name="volume" value="0.75"/>  
    <set name="tag">  
        <string>kotlin</string>  
        <string>android</string>  
    </set>  
</map>
```

XML

Pattern: incapsulare in una classe

Spargere chiamate a getString/putBoolean per il codice è una cattiva idea. Centralizziamo l'accesso in una classe dedicata.

```
class UserPreferences(context: Context) {  
  
    private val prefs = context.applicationContext  
        .getSharedPreferences("user_prefs", Context.MODE_PRIVATE)  
  
    var username: String?  
        get() = prefs.getString(KEY_USERNAME, null)  
        set(value) = prefs.edit { putString(KEY_USERNAME, value) }  
  
    var darkMode: Boolean  
        get() = prefs.getBoolean(KEY_DARK_MODE, false)  
        set(value) = prefs.edit { putBoolean(KEY_DARK_MODE, value) }  
  
    fun logout() = prefs.edit { clear() }  
  
    private companion object {  
        const val KEY_USERNAME = "username"  
        const val KEY_DARK_MODE = "dark_mode"  
    }  
}
```

Kotlin

Vantaggi e uso del Repository

Una volta incapsulato l'accesso, il resto del codice resta pulito e testabile.

```
val userPrefs = UserPreferences(this)

// Lettura: come una qualunque proprietà
val nome = userPrefs.username ?: "Ospite"
binding.helloText.text = "Ciao, $nome"

// Scrittura: assegnamento
userPrefs.darkMode = true

// Logout
binding.logoutBtn.setOnClickListener {
    userPrefs.logout()
}
```

Kotlin

Cosa ci abbiamo guadagnato

- Chiavi in un solo posto: niente typo magici
- Tipo di ritorno chiaro: niente cast
- Codice client più leggibile
- Facile da mockare nei test
- Migrazione futura più semplice (es. DataStore)

Avanzato: property delegation

Possiamo creare un delegato Kotlin per evitare di ripetere il pattern getter/setter per ogni preferenza.

```
class BoolPref(
    private val prefs: SharedPreferences,
    private val key: String,
    private val default: Boolean
) : ReadWriteProperty<Any?, Boolean> {

    override fun getValue(
        thisRef: Any?, p: KProperty<*>
    ): Boolean = prefs.getBoolean(key, default)

    override fun setValue(
        thisRef: Any?, p: KProperty<*>,
        value: Boolean
    ) { prefs.edit { putBoolean(key, value) } }
}
```

Kotlin

```
// Uso
class UserPreferences(
    ctx: Context
) {
    private val p = ctx
        .getSharedPreferences(
            "user_prefs",
            Context.MODE_PRIVATE
        )

    var darkMode by BoolPref(
        p, "dark_mode", false
    )

    var notifiche by BoolPref(
        p, "notifiche", true
    )
}
```

Kotlin

Errori comuni da evitare

✗ Dimenticare `apply()/commit()`

Senza chiamarli, l'editor accumula modifiche che non vengono mai salvate.

✗ Usare `commit()` sul main thread

Blocca la UI fino al completamento I/O. Usa `apply()` salvo necessità.

✗ Salvare oggetti complessi come stringa concatenata

Per dati strutturati: serializza in JSON e leggi con Gson/Moshi/kotlinx.serialization.

✗ Non usare la application context

Usare l'Activity context puo' creare leak. Sempre `context.applicationContext`.

✗ Stringhe magiche per le chiavi

Centralizza le chiavi in costanti / companion object per evitare refusi.

✗ Salvare dati sensibili in chiaro

Password, token: usa `EncryptedSharedPreferences` o un keystore.

Sicurezza: cosa NON salvare

Le SharedPreferences standard salvano in chiaro. Su un dispositivo rooted o un backup non cifrato il file XML è leggibile da chiunque abbia accesso al filesystem.

NON ADATTO

- Password in chiaro
- Token di autenticazione (JWT, OAuth)
- Numeri di carta di credito
- Chiavi private
- Dati personali sensibili (sanitari, ecc.)

ADATTO

- Preferenze UI (tema, lingua, font)
- Flag di onboarding ("primo avvio")
- Username pubblico
- Cache di piccoli dati non riservati
- Posizione (offset) di scorrimento ricordata

EncryptedSharedPreferences

La libreria `androidx.security` fornisce un'implementazione di `SharedPreferences` che cifra automaticamente chiavi e valori usando il Keystore di Android.

```
// build.gradle.kts
// implementation("androidx.security:security-crypto:1.1.0-alpha06")

val masterKey = MasterKey.Builder(context)
    .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
    .build()

val securePrefs = EncryptedSharedPreferences.create(
    context, "secure_prefs", masterKey,
    EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
    EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM
)

// Stessa identica API di SharedPreferences
securePrefs.edit { putString("auth_token", "xyz...") }
val token = securePrefs.getString("auth_token", null)
```

Kotlin



Stessa API: il refactor di un file da `SharedPreferences` a `EncryptedSharedPreferences` cambia solo la creazione.

I limiti delle SharedPreferences

Pur essendo comode e veloci da imparare, le SharedPreferences hanno problemi noti che diventano evidenti in app non banali.

API non sicura sui tipi

Si può salvare un `Int` e leggerlo come `String` per errore: il tipo non è verificato a compile time.

Niente coroutine / Flow

L'API è sincrona. Per osservare i cambiamenti serve il listener, che non si integra con Kotlin Flow.

`apply()` non segnala errori

Se la scrittura su disco fallisce, l'app non lo sa. `commit()` segnala ma blocca.

Carica tutto il file in memoria

Anche se ci serve una sola chiave, il file viene letto interamente la prima volta.

Modifiche concorrenti delicate

Più Editor sullo stesso file possono produrre comportamenti inattesi.

Race conditions in startup

Accedere a prefs durante `onCreate` può causare un blocco I/O sul main thread.

L'alternativa moderna: Jetpack DataStore

Google raccomanda DataStore come sostituto delle SharedPreferences nelle nuove app. Offre API basate su coroutine e Flow.

```
// implementation(  
//   "androidx.datastore:datastore-preferences:1.1.1"  
// )  
  
val Context.dataStore by  
    preferencesDataStore(name = "user_prefs")  
  
// Lettura come Flow (osservabile)  
val DARK_MODE = booleanPreferencesKey("dark_mode")  
  
val darkModeFlow: Flow<Boolean> = context.dataStore  
    .data  
    .map { prefs -> prefs[DARK_MODE] ?: false }  
  
// Scrittura: suspend, sicura sui tipi  
suspend fun setDarkMode(enabled: Boolean) {  
    context.dataStore.edit { prefs ->  
        prefs[DARK_MODE] = enabled  
    }  
}
```

Kotlin

Cosa cambia rispetto a SharedPreferences

- Asincrono via coroutine
- Osservabile come Flow
- Tipi sicuri (preferencesKey)
- Errori segnalati come exception
- API consistente e moderna

Migrazione automatica a DataStore

DataStore offre un meccanismo di migrazione: i dati di un file SharedPreferences esistente vengono spostati in modo trasparente alla prima lettura.

```
val Context.dataStore by preferencesDataStore(  
    name = "user_prefs",  
    produceMigrations = { ctx ->  
        listOf(  
            // Migra il file SharedPreferences "user_prefs" esistente  
            SharedPreferencesMigration(ctx, "user_prefs")  
        )  
    }  
)  
  
// Alla prima lettura, le chiavi vengono copiate automaticamente.  
// Il file XML originale viene poi cancellato.
```

Kotlin



Strategia consigliata

Nuove app: partire direttamente da DataStore. App esistenti: migrare incrementalmente, file per file, mantenendo l'API del Repository.

Cheat-sheet finale

Le operazioni essenziali su SharedPreferences in un colpo d'occhio.

Kotlin

```
// 1. Ottenere l'istanza
val prefs = context.getSharedPreferences("user_prefs", Context.MODE_PRIVATE)

// 2. Leggere (con default)
val nome     = prefs.getString("username", null)
val tema     = prefs.getBoolean("dark_mode", false)
val livello  = prefs.getInt("livello", 0)

// 3. Scrivere (idiomatico Kotlin)
prefs.edit {
    putString("username", "alice")
    putBoolean("dark_mode", true)
    putInt("livello", 5)
}

// 4. Operazioni aggiuntive
prefs.contains("username")           // Boolean
prefs.edit { remove("username") }   // rimuove una chiave
prefs.edit { clear() }               // rimuove tutto
val tutte: Map<String, *> = prefs.all
```